FINSIGHT BACKEND

Authentication, Explained From Zero

How JWT access & refresh tokens, httpOnly cookies, and Google sign-in work in this NestJS API —
every architecture decision, every file, every flow.

PROJECT

AI Expense & Personal Finance Dashboard (FinSight)

SCOPE

`fin-sight-backend/` **— backend only**

STACK

NestJS 11 · TypeScript 5.9 · TypeORM 1.1 · PostgreSQL · Passport · JWT · bcrypt · Google OAuth 2.0

WRITTEN FOR

Someone who has never built authentication before

> **How to read this.** Parts 1–3 are concepts — read them even if you want to skip ahead, because everything later assumes them. Part 6 walks through every file in the order the code actually runs. Part 7 traces the six real request flows end to end — including Sign in with Google. If you only have ten minutes, read Part 3 and Part 7.

Contents

1 The Problem Authentication Solves

Why a server cannot simply remember who you are.

HTTP forgets you between every request

When your browser talks to a server, each request is a completely separate conversation. The server answers, closes the connection, and forgets everything. This is called being **stateless**. It is what makes the web scale — but it creates an obvious problem.

You type your email and password once. Then you click "My expenses". That click is a brand new request, and as far as the server is concerned, it comes from a total stranger. Nothing in it says who you are.

The server has no memory linking request 2 back to request 1.

So every request after login has to *carry proof* of who you are. The whole field of authentication is about the design of that proof. Two questions decide everything:

- **What does the proof look like?** A random ID? A signed document?
- **Where does the server check it?** In a database lookup, or by doing maths on the proof itself?

The two classic answers

Approach	How it works	The catch
Sessions	After login the server generates a random string, stores it in a table alongside your user ID, and sends it to the browser as a cookie. On every later request it looks that string up in the table.	Every single request costs a database lookup. And every server instance needs access to the same session store, which becomes a shared bottleneck the moment you run more than one server.
Tokens (JWT)	After login the server builds a small signed document containing your user ID, and hands it over. On every later request it verifies the signature with a secret key. If the signature checks out, the contents are trusted. No database lookup required.	Because there is nothing stored, there is nothing to delete. A token stays valid until it expires — you cannot cancel it. This is *the* problem, and Part 3 is entirely about solving it.

This project uses the token approach, and then adds a targeted fix for its one real weakness. That fix is the access + refresh split.

WHY TOKENS HERE

FinSight's backend is a plain JSON API consumed by a separate React frontend running on a different origin. Cookies across origins mean CORS credentials, `SameSite` rules, and CSRF defences. A bearer token in a header sidesteps all of that, and it is the same mechanism a future mobile app would use. For a small API with a separate SPA frontend, tokens are the path of least resistance.

2 Vocabulary You Need First

Six terms. Everything after this assumes you know them.

2.1 Hashing — a one-way door

A hash function takes any input and produces a fixed-length scramble. The critical property: it only runs *forwards*. Given the scramble, there is no practical way to recover the input.

```
"password123"  ──hash──▶  "$2b$10$N9qo8uLOickgx2ZMRZoMye1J8..."
"password124"  ──hash──▶  "$2b$10$KIXqFPz9lRs8sN0bWVfg9uYc2P..."   ← totally different
```

This is why we never store passwords. We store the hash. When you log in, we hash what you typed and compare the two hashes. If our database leaks, the attacker gets scrambles, not passwords.

Salt — why two identical passwords hash differently

If hashing were purely deterministic, every user with the password `password123` would have the identical hash in the database. An attacker could hash the top million common passwords once and match them all in bulk — a "rainbow table" attack.

A **salt** is random data mixed in before hashing, and stored alongside the result. Same password, different salt, different hash. The attacker now has to attack each user separately. bcrypt generates and embeds the salt for you — it is part of that `$2b$10$N9qo8uLO…` string.

Why bcrypt specifically, and what "10 rounds" means

General-purpose hashes like SHA-256 are designed to be *fast*. That is the wrong property for passwords: fast means an attacker with a GPU can try billions of guesses per second. bcrypt is deliberately, tunably *slow*.

The cost factor — `SALT_ROUNDS = 10` in our code — is an exponent. Each increment doubles the work. 10 rounds takes roughly 50–100 ms on normal hardware. Unnoticeable when one real person logs in; ruinous when someone is trying to brute-force a stolen database.

2.2 JWT — a signed, readable ID card

A **JSON Web Token** is three Base64 chunks joined by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiJhYmMtMTIzIiwiZW1haWwiOiJ1c2VyQGV4LmNvbSJ9 . 4pV8f-2mQ0x…
└──────────── HEADER ────────────────┘ └──────────────────── PAYLOAD ──────────────────────────┘ └── SIGNATURE ──┘
```

Header	Which algorithm signed this. Ours: `{"alg":"HS256","typ":"JWT"}`
Payload	The actual data — called **claims**. Ours: `{"sub":"…","email":"…","iat":…,"exp":…}`
Signature	HMAC-SHA256 of `header.payload` using the server's secret key.

> **THE SINGLE MOST MISUNDERSTOOD THING ABOUT JWTS**
>
> The payload is **Base64-encoded, not encrypted**. Anyone holding the token can decode and read it — paste one into `jwt.io` and you will see your own email in plain text. The signature guarantees the payload has not been *changed*; it guarantees nothing about secrecy.
>
> Practical rule: never put anything in a JWT you would not be comfortable printing on the front page of a newspaper. No passwords, no card numbers, no private notes. Our payload holds only a user ID and an email — both of which the user already knows.

Why the signature cannot be faked

Suppose an attacker decodes their token, changes `"sub"` to someone else's user ID, and re-encodes it. The payload changed, so the correct signature changed too. To compute the new correct signature they would need the server's secret key. They do not have it. The server recomputes the signature on arrival, sees a mismatch, and rejects the token.

This is the entire trick that makes stateless authentication possible: the server does not need to remember issuing the token, because it can re-derive whether it *would have* issued it.

Registered claims we rely on

Claim	Meaning	How we use it
`sub`	Subject — who the token is about	The user's UUID. This is *the* identity claim.
`iat`	Issued at (Unix seconds)	Set automatically. Note: **second** precision — this causes a real problem, see §8.6.
`exp`	Expires at (Unix seconds)	Set from `expiresIn`. Passport rejects expired tokens before our code runs.
`jti`	JWT ID — a unique identifier	A random UUID we attach to refresh tokens only, to guarantee uniqueness.

2.3 Bearer token — "whoever holds this, is them"

The token travels in a standard HTTP header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhYmMtMTIz...
```

The word "Bearer" is literal and it is a warning. There is no second factor, no proof of possession, no binding to a device. The server trusts whoever presents the token. A stolen token *is* the user until it expires. Everything in Part 3 exists because of that sentence.

2.4 Authentication vs Authorization

Term	Question	In this project
Authentication (authn)	Who are you?	Everything in this document. Verifying the password, issuing and checking tokens.
Authorization (authz)	What are you allowed to do?	Not built. There are no roles or permissions yet — every logged-in user has identical rights. See Part 13.

2.5 Revocation — the word that shapes this whole design

To **revoke** a credential is to cancel it before its natural expiry. With sessions this is trivial: delete the row. With a plain JWT it is impossible — the token is valid because the maths says so, and the maths does not consult your database.

So "log out" on a pure JWT system is a lie. The frontend deletes its copy of the token, but the token itself keeps working until `exp` passes. If someone captured it, they are still logged in. Part 3 is how we make logout real.

2.6 Repository, entity, ORM

An **entity** is a TypeScript class that maps to a database table — one class, one table; one property, one column. A **repository** is the object that reads and writes those rows for you (`save` , `findOne` , `update`). The library that wires classes to tables is the **ORM** — here, TypeORM. You write TypeScript; it writes SQL.

3 The Access + Refresh Token Design

The central architecture decision. If you read one part, read this one.

The trade-off that forces the design

Pick a lifetime for your token. Both directions hurt:

If the token is…	Good	Bad
Long-lived (say, 30 days)	The user logs in once a month. Pleasant.	A token stolen on day 1 works for 29 more days, and you cannot cancel it. One leaked token is a month-long breach.
Short-lived (say, 15 minutes)	A stolen token is nearly worthless — it dies on its own, fast.	The user is thrown back to the login screen every 15 minutes. Nobody accepts that.

Both requirements are legitimate and they point in opposite directions. The resolution is to stop trying to satisfy both with one credential, and issue **two tokens with different jobs**.

Two tokens, two jobs

Two credentials, deliberately different in exposure and lifetime.

The insight is about **exposure**. The access token is everywhere — in every request, every log line, every proxy, every browser devtools panel. So it is short-lived: high exposure, low value. The refresh token appears in exactly one request, to one endpoint, a handful of times per week. Low exposure, so it can afford to be valuable.

> **THE PAYOFF**
>
> **The user logs in once a week. A stolen access token dies in fifteen minutes. And because the refresh token is checked against the database, we can cancel it — which means logout is real.**

The part most tutorials skip: the refresh token is stateful

Here is the subtlety. The access token is pure JWT — verified by maths alone, never touching the database. That is what makes it fast.

The refresh token is *also* a JWT, but we do not trust the maths alone. We additionally store a hash of it on the user's row, and on every refresh we check that the token presented matches the one we last handed out.

users table

id	email	password	hashedRefreshToken
abc-123	user@example.com	$2b$10$N9qo8u…	7f83b1657ff1fc53b92dc1815…

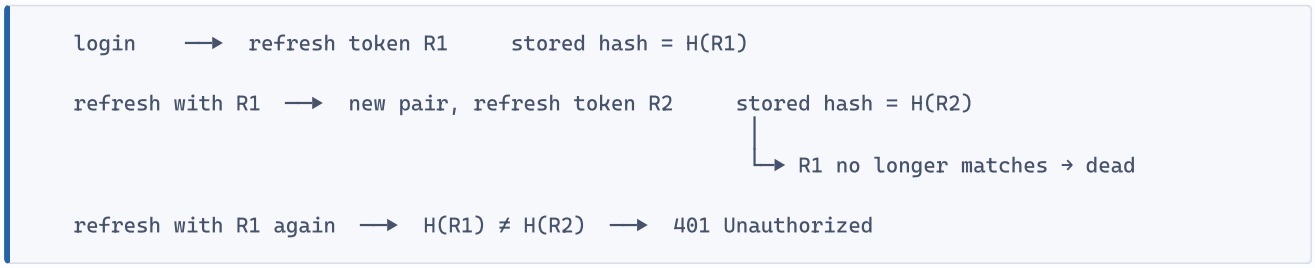
▲ bcrypt of the password

▲ SHA-256 of the refresh token
NULL = logged out, revoked

This deliberately gives up a little of JWT's statelessness — but only on the one endpoint that is called rarely. We pay a database lookup once a week instead of on every request, and in exchange we get revocation back.

Rotation: each refresh invalidates the last

Every call to `/auth/refresh` issues a *new* refresh token and overwrites the stored hash. The token you just used is now dead.



Rotation. A refresh token is single-use by construction.

Why this matters: it shrinks the window in which a stolen refresh token is useful. The thief must use it before the real user's app next refreshes — otherwise their copy is already dead.

> **HONEST LIMITATION**
>
> We detect that an old token no longer matches, but we do not *react* to it. A full implementation treats "someone presented a rotated-away token" as evidence of theft and kills every session for that user. Ours simply returns 401. See Part 13 — this is a known, deliberate gap, not an oversight.

Why two different secrets

The access token is signed with `JWT_ACCESS_SECRET`. The refresh token is signed with `JWT_REFRESH_SECRET`. These must be different values, and this is not decoration.

If both used one secret, the two tokens would be cryptographically indistinguishable. A refresh token — the long-lived, valuable one — would sail through the access-token guard and work as a 7-day access token on every protected endpoint. The entire short-lifetime property would evaporate.

With separate secrets, presenting a refresh token to a protected route fails at the signature check. There is a test asserting exactly this: "*refuses an access token on the refresh route*".

Where these tokens are stored (as this project actually does it)

Storage	Survives reload	Trade-off
In-memory JS variable	No	Immune to XSS theft, but the user is logged out on every refresh of the page.
`localStorage`	Yes	Simple and common. But any XSS on your site can read it. This is what most tutorials do.
`httpOnly` cookie	Yes	JavaScript cannot read it, so XSS cannot steal it — but now you must handle CSRF and cross-origin cookie rules.

This project uses the strongest common split, and the backend enforces it:

- **Access token → JSON response body.** The frontend keeps it in a memory variable. Short-lived, so losing it on reload only costs one silent refresh.
- **Refresh token → `httpOnly` cookie**, set by the server. JavaScript can never read it, so an XSS bug cannot steal it. The browser sends it back automatically, and only to `/auth/*`.

> **THE BACKEND MAKES THE SAFE CHOICE FOR YOU**
>
> The refresh token is **not** in any JSON body. It exists only inside the `httpOnly` cookie. That means even a frontend author who wanted to stash it in `localStorage` could not — it is not reachable from JavaScript at all. The one exception is the Google flow, where the access token comes back on the URL fragment instead of a body; the refresh token still travels only by cookie. Part 6 and Part 7 show the exact mechanics.

4 NestJS Building Blocks

Nine concepts. Once these click, the code reads itself.

NestJS organises code into small, single-purpose pieces that it wires together for you. If you have seen Angular, the shape is familiar. If you have seen Spring Boot, it is nearly identical in spirit.

Piece	One-line job	In our code
Module	A box grouping related things and declaring what it needs and shares.	`AuthModule`, `UsersModule`, `AppModule`
Controller	Maps URLs to functions. Should contain no logic.	`AuthController` — five routes, each one line
Service	Where the actual thinking happens. Knows nothing about HTTP.	`AuthService`, `UsersService`
DTO	Describes and validates the shape of data crossing the boundary.	`CreateUserDto`, `LoginDto`, `TokensDto`
Entity	A class that maps to a database table.	`User` → the `users` table
Pipe	Transforms or validates input before the controller sees it.	One global `ValidationPipe`
Guard	Yes/no gatekeeper. Runs before the controller; can block the request.	`JwtAuthGuard`, `JwtRefreshGuard`
Strategy	Passport plugin defining *how* a credential is verified.	`JwtStrategy`, `JwtRefreshStrategy`
Decorator	The `@Something()` syntax — attaches metadata Nest reads at startup.	`@Controller`, `@Post`, `@CurrentUser`

4.1 Dependency injection, the one concept to actually understand

Look at this constructor:

```
@Injectable()
export class AuthService {
  constructor(
    private readonly usersService: UsersService,
    private readonly jwtService: JwtService,
    private readonly configService: ConfigService,
  ) {}
}
```

Notice what is *absent*: nowhere do we write `new UsersService(...)`. We declare what we need, and Nest constructs and supplies it. That is dependency injection.

It reads like magic but the mechanism is mundane. `@Injectable()` combined with TypeScript's `emitDecoratorMetadata` makes the compiler record the constructor's parameter types into the compiled JavaScript. At startup Nest reads that metadata, sees `UsersService` is needed, finds who provides it, builds it (recursively building *its* dependencies), and passes it in. One shared instance per module — a singleton.

> **WHY THIS EARNS ITS KEEP**
>
> It is what makes the tests possible. In `auth.service.spec.ts` we hand `AuthService` a fake users service and a fake config, and it cannot tell the difference — because it never asked for the real ones by name, it only declared a shape. Sixteen tests run with no database because of this one property.

4.2 The request pipeline — what runs, in what order

Guards run before pipes. Order matters — see the note below.

> **A DETAIL WORTH INTERNALISING**
>
> **Guards run before pipes.** On `POST /auth/login` there is no guard, so validation runs first and a malformed body gets a clean 400. But on `POST /auth/refresh` the guard runs first — an invalid token is rejected with 401 before the body is ever inspected. That ordering is correct: there is no reason to spend effort validating input from a caller you have not authenticated.

4.3 What Passport actually does

Passport is a small library with one idea: a **strategy** is a plugin that knows how to verify one kind of credential. There are strategies for passwords, Google, GitHub, API keys, and — the ones we use — JWTs.

Each strategy has a name. `JwtStrategy` registers as `'jwt'`; `JwtRefreshStrategy` registers as `'jwt-refresh'`. A guard is just the thing that says which name to run:

```
export class JwtAuthGuard    extends AuthGuard('jwt')         {}
export class JwtRefreshGuard extends AuthGuard('jwt-refresh') {}
```

The division of labour inside a strategy is worth being precise about:

Done by Passport, automatically	Done by our `validate()`
Extract the token from the `Authorization` header	Look the user up in the database
Verify the signature against the secret	Confirm the user still exists
Check `exp` and reject expired tokens	Decide what gets attached to `request.user`

By the time `validate()` runs, the token is already proven authentic and unexpired. Whatever `validate()` returns becomes `request.user`. If it throws, the request gets a 401.

5 The Folder Structure, and Why

What a professional NestJS codebase looks like, and the reasoning.

```
fin-sight-backend/
├── .env                        ← real secrets. gitignored. never committed.
├── .env.example                ← the template, safe to commit
├── src/
│   ├── main.ts                 ← entry point: global setup, then start listening
│   ├── app.module.ts           ← the root module: database + feature modules
│   ├── app.controller.ts       ← trivial health route
│   ├── app.service.ts
│   │
│   ├── common/                 ← shared across features, belongs to none
│   │   ├── decorators/
│   │   │   └── current-user.decorator.ts
│   │   └── interfaces/
│   │       └── jwt-payload.interface.ts
│   │
│   └── modules/                ← one folder per feature
│       ├── auth/
│       │   ├── auth.module.ts
│       │   ├── auth.controller.ts        ← HTTP layer (also sets the cookie)
│       │   ├── auth.service.ts           ← business logic
│       │   ├── auth.service.spec.ts      ← unit tests
│       │   ├── refresh-cookie.ts         ← cookie name + options, one place
│       │   ├── dto/
│       │   │   ├── login.dto.ts
│       │   │   ├── tokens.dto.ts
│       │   │   └── auth-response.dto.ts
│       │   ├── guards/
│       │   │   ├── jwt-auth.guard.ts
│       │   │   ├── jwt-refresh.guard.ts
│       │   │   └── google-auth.guard.ts
│       │   └── strategies/
│       │       ├── jwt.strategy.ts
│       │       ├── jwt-refresh.strategy.ts   ← reads the refresh token from the cookie
│       │       └── google.strategy.ts        ← Sign in with Google
│       │
│       └── users/
│           ├── users.module.ts
│           ├── users.service.ts           ← the only code touching the users table
│           ├── dto/create-user.dto.ts
│           └── entities/user.entity.ts     ← password nullable, provider column
└── test/
    ├── app.e2e-spec.ts
    └── auth.e2e-spec.ts       ← full HTTP tests, no database needed
```

The three rules this layout follows

1. Group by feature, not by technical type

A beginner's instinct is to make top-level folders called `controllers/`, `services/`, `entities/`. It looks tidy and it is a trap. Changing one feature then means touching four scattered folders, and nothing tells you which

files belong together.

Grouping by feature means everything about authentication is in one folder. You can read it, review it, or delete it as a unit. As the app grows — `modules/expenses/`, `modules/budgets/` — each new feature is an isolated addition rather than four more files sprinkled into four growing piles.

2. `common/` is for things owned by nobody

The `CurrentUser` decorator and the `JwtPayload` interface are used by both auth and (soon) every other feature. Putting them under `auth/` would mean `modules/expenses/` importing from `modules/auth/` just to read a type — a dependency that implies a relationship which does not exist.

> **THE DISCIPLINE THAT KEEPS THIS HONEST**
>
> `common/` is where shared code goes, which makes it exactly where junk accumulates. The rule: something moves into `common/` when a *second* feature genuinely needs it — never in anticipation. A `common/` folder full of single-use helpers is a code smell, not a sign of good architecture.

3. One module owns its table

`UsersService` is the only class in the codebase that touches the `users` table. `AuthService` never injects a repository; it asks `UsersService`.

The benefit shows up the first time a rule changes. Suppose emails must be case-insensitive. There is exactly one place that normalises them (`normalizeEmail` in `users.service.ts`) and every caller inherits the fix. Had three services each written their own `findOne({ where: { email } })`, you would be hunting for all three and finding two.

Why relative imports and not path aliases

The previous version of this code imported like `import { UsersService } from '@users/users.service'`. Clean-looking, and it did not work.

Path aliases are a *TypeScript* feature. The compiler resolves them for type-checking, then emits the JavaScript with the alias string untouched. Node has no idea what `@users` means, so the app crashes on startup unless you add a runtime resolver such as `tsconfig-paths`. That was not configured, and the alias was not in `tsconfig.json` at all — so it failed at both compile and run time.

Relative imports (`../users/users.service`) are marginally uglier and work everywhere with zero configuration. For a project this size that is the right trade.

6 Every File, Explained

In the order the code runs, not alphabetically.

6.1 `src/main.ts` — the entry point

src/main.ts

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Populates request.cookies, so the refresh strategy can read the refresh token.
  app.use(cookieParser());

  // Without this the class-validator rules on the DTOs never run.
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,            // drop properties the DTO does not declare
      forbidNonWhitelisted: true, // ...and reject the request if it sent any
      transform: true,            // turn the plain body into a real DTO instance
    }),
  );

  // credentials:true lets the browser send/receive the httpOnly refresh cookie.
  app.enableCors({ origin: true, credentials: true });

  const config = new DocumentBuilder()
    .setTitle('FinSight Backend')
    .setDescription('AI Expense & Personal Finance Dashboard API')
    .setVersion('1.0')
    .addBearerAuth()
    .build();

  SwaggerModule.setup('api/docs', app, SwaggerModule.createDocument(app, config));

  const port = process.env.PORT ?? 3000;
  await app.listen(port);
}

void bootstrap();
```

Four global concerns, set up once.

> **WHY COOKIEPARSER AND CREDENTIALS MATTER HERE**
>
> `app.use(cookieParser())` is what fills `request.cookies` — without it the refresh strategy would find nothing. And `credentials: true` on CORS is what allows the browser to send the httpOnly cookie back on cross-origin calls. Both are new since the token pair moved into a cookie; miss either and refresh silently stops working.

The ValidationPipe is not optional

This is the single most important line in the file. Without it, every `@IsEmail()` and `@MinLength(8)` in the DTOs is inert decoration. You would believe you had validation; you would have none. The original code had all the decorators and no pipe.

whitelist	Silently strips any property not declared in the DTO. Send `{ email, password, isAdmin: true }` and `isAdmin` is removed before your code sees it. This is a real defence against mass-assignment attacks.
forbidNonWhitelisted	Escalates stripping to rejecting — respond 400 instead. Louder, and much better during development, because a client typo becomes a visible error rather than a silently ignored field.
transform	Converts the plain parsed JSON into an actual instance of the DTO class, so type conversions and defaults apply.

`void bootstrap()`

`bootstrap()` returns a promise nobody awaits. The `void` operator is an explicit statement of "yes, I know, and it is intentional" — it satisfies the `no-floating-promises` lint rule without a suppression comment.

6.2 `src/app.module.ts` — the root

src/app.module.ts

```
@Module({
  imports: [
    ConfigModule.forRoot({
      isGlobal: true,       // so any module can inject ConfigService without importing this
      envFilePath: '.env',
    }),
    TypeOrmModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (configService: ConfigService) => ({
        type: 'postgres',
        host: configService.get<string>('DATABASE_HOST', 'localhost'),
        port: configService.get<number>('DATABASE_PORT', 5432),
        username: configService.getOrThrow<string>('DATABASE_USERNAME'),
        password: configService.getOrThrow<string>('DATABASE_PASSWORD'),
        database: configService.getOrThrow<string>('DATABASE_NAME'),
        entities: [__dirname + '/**/*.entity{.ts,.js}'],
        synchronize: configService.get<string>('NODE_ENV') !== 'production',
      }),
    }),
    AuthModule,
    UsersModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

`forRootAsync` and why it is not `forRoot`

The database settings live in `.env`, which means we cannot write them as a literal object — we need `ConfigService` to read them, and `ConfigService` is itself something Nest has to construct. `forRootAsync` lets us say: build `ConfigService` first, hand it to this factory, use the returned object as the configuration.

`get` vs `getOrThrow`

A deliberate distinction. `get('DATABASE_HOST', 'localhost')` has a sane fallback — most people run Postgres locally. But there is no sane default for a password, so `getOrThrow` is used: if it is missing the app refuses to start, loudly, at boot.

> **FAIL FAST, AT STARTUP**
>
> Crashing on boot with "DATABASE_PASSWORD is not defined" is enormously better than starting successfully and failing on the first user's request at 2am. The same reasoning applies to the JWT secrets, which also use `getOrThrow`. A misconfigured app should not be able to pretend it is healthy.

`synchronize` — convenient now, dangerous later

When true, TypeORM inspects your entity classes on every startup and alters the database to match. Add a property, restart, the column appears. Excellent for development.

> **NEVER TRUE IN PRODUCTION**
>
> `synchronize` will happily **drop a column** — and all its data — because you renamed a property. In production you use migrations: reviewed, versioned, reversible SQL. That is why the value is bound to `NODE_ENV !== 'production'` rather than hard-coded `true`.

6.3 `src/modules/users/entities/user.entity.ts`

`src/modules/users/entities/user.entity.ts`

```
export type AuthProvider = 'local' | 'google';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id!: string;

  @Column({ type: 'varchar', unique: true })
  email!: string;

  /**
   * bcrypt hash — the plain password is never stored or logged.
   * Nullable because Google users have no password.
   */
  @Column({ type: 'varchar', nullable: true })
  password?: string | null;

  /** 'local' = email + password, 'google' = signed in via Google. */
  @Column({ type: 'varchar', default: 'local' })
  provider!: AuthProvider;

  @Column({ type: 'varchar', nullable: true })
  name?: string;

  @Column({ type: 'varchar', nullable: true })
  hashedRefreshToken?: string | null;

  @Column({ type: 'timestamp', nullable: true })
  lastLoginAt?: Date | null;

  @CreateDateColumn() createdAt!: Date;
  @UpdateDateColumn() updatedAt!: Date;
}
```

@Entity('users')	This class is the `users` table.
uuid, not auto-increment	Sequential integer IDs leak business information (user #1051 tells an observer how many users you have) and invite enumeration — try `/users/1`, `/users/2`. UUIDs are unguessable and can be generated client-side or across services without collision.
unique: true on email	A database-level constraint. Application checks can lose a race between two simultaneous signups; the database cannot. This is the real guarantee.
password nullable	Google users authenticate through Google and never set a password, so the column has to allow `NULL`. Password login rejects such accounts (see 6.4).
provider	Records how the account signs in. Defaults to `'local'`; Google accounts are `'google'`. Makes the account type explicit rather than guessed from whether the password is null.
hashedRefreshToken	The heart of Part 3. Nullable because `NULL` is the representation of "logged out — revoked".
the `!` suffix	TypeScript's definite-assignment assertion: "I know it is not assigned in a constructor; TypeORM fills it in." Purely a compiler concern.

6.4 `src/modules/users/users.service.ts`

The only class permitted to touch the `users` table.

`src/modules/users/users.service.ts (top)`

```
const SALT_ROUNDS = 10;

/** A throwaway hash, computed once at startup, only used to burn time on unknown emails. */
const DUMMY_HASH = bcrypt.hashSync('not-a-real-password', SALT_ROUNDS);

function normalizeEmail(email: string): string {
  return email.trim().toLowerCase();
}

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly usersRepository: Repository<User>,
  ) {}

  async create(createUserDto: CreateUserDto): Promise<User> {
    const user = this.usersRepository.create({
      email: normalizeEmail(createUserDto.email),
      password: await bcrypt.hash(createUserDto.password, SALT_ROUNDS),
      provider: 'local',
      name: createUserDto.name,
    });

    return this.usersRepository.save(user);
  }

  /** Google sign-in: find the user by email, or create one (no password). */
  async findOrCreateGoogleUser(googleUser: {
    email: string;
    name?: string;
  }): Promise<User> {
    const existing = await this.findByEmail(googleUser.email);
    if (existing) return existing;   // reuse — the person owns this email

    const user = this.usersRepository.create({
      email: normalizeEmail(googleUser.email),
      password: null,
      provider: 'google',
      name: googleUser.name,
    });
    return this.usersRepository.save(user);
  }
```

`@InjectRepository(User)` asks Nest for TypeORM's repository for the `User` entity — the object that can `save`, `findOne`, `update` rows in `users`.

Note the ordering in `create`: the password is hashed *before* the object is built. The plain password never exists inside an entity, so it can never be accidentally persisted or serialised.

> **WHY NORMALIZEEMAIL EXISTS**
>
> `Jawad@Example.com` and `jawad@example.com` are the same mailbox to every real mail provider, but different strings to Postgres — so the `unique` constraint would happily allow both as separate accounts, and the user would be baffled when their password "stopped working". Lowercasing on both write and read makes the constraint behave the way people expect.

The timing-attack defence

`src/modules/users/users.service.ts (validateCredentials)`

```
async validateCredentials(email: string, password: string): Promise<User | null> {
  const user = await this.findByEmail(email);

  // No user, OR a Google account with no password: run the dummy compare so the
  // timing matches the real path, then reject.
  if (!user || !user.password) {
    await bcrypt.compare(password, DUMMY_HASH);
    return null;
  }

  const passwordMatches = await bcrypt.compare(password, user.password);
  return passwordMatches ? user : null;
}
```

The extra `|| !user.password` matters once Google accounts exist: they have a null password, so trying to log into one via the password route must fail — and fail with the same timing as any other failure, so it does the dummy compare too. A Google user cannot be brute-forced through the password door.

That `bcrypt.compare` against a dummy hash looks pointless. It is not, and it is worth understanding why.

Without it, the two failure paths take very different amounts of time:

Scenario	Work done	Response time
Email does not exist	One quick indexed lookup, then return	~2 ms
Email exists, wrong password	Lookup *plus* a full bcrypt comparison	~80 ms

Both return the same 401 and the same message — but an attacker does not need to read the message. They time the response. A 2 ms reply means "no such account"; an 80 ms reply means "that account exists, keep guessing". They can now harvest your entire user list without ever logging in. This class of bug is called a **timing side-channel**, and it is invisible in code review unless you are looking for it.

Running bcrypt against a throwaway hash makes both paths cost roughly the same, and the signal disappears.

6.5 The DTOs

A DTO — Data Transfer Object — is a class describing the shape of data crossing the API boundary. It serves three purposes at once: it is the TypeScript type, the validation rules, and the Swagger documentation.

`src/modules/users/dto/create-user.dto.ts`

```
export class CreateUserDto {
  @ApiProperty({ description: 'User email', example: 'user@example.com' })
  @IsEmail({}, { message: 'Please provide a valid email address' })
  email!: string;

  @ApiProperty({ description: 'User password (at least 8 characters)', minLength: 8 })
  @IsString()
  @MinLength(8, { message: 'Password must be at least 8 characters long' })
  password!: string;

  @ApiPropertyOptional({ description: 'User name', example: 'John Doe' })
  @IsOptional()
  @IsString()
  name?: string;
}
```

Two families of decorator, doing different jobs:

- `@IsEmail`, `@MinLength`, `@IsOptional` — from **class-validator**. These are the rules the `ValidationPipe` enforces at runtime.
- `@ApiProperty` — from **@nestjs/swagger**. Purely documentation; generates the interactive API page at `/api/docs`.

> **WHY THE VALIDATION LIVES HERE AND NOT IN THE SERVICE**
>
> Input validation belongs at the boundary — the first place untrusted data arrives. By the time `AuthService.register()` is called, the email is already known to be well-formed and the password already known to be long enough. The service is free to concentrate on business rules, and there is exactly one place to look for input rules.

The response DTOs

`src/modules/auth/dto/tokens.dto.ts & auth-response.dto.ts`

```
export class TokensDto {
  accessToken!: string;
  refreshToken!: string;
}

class UserSummaryDto {
  id!: string;
  email!: string;
  name?: string;
}

/** What register and login return: the tokens plus the safe parts of the user. */
export class AuthResponseDto extends TokensDto {
  user!: UserSummaryDto;
}
```

The split mirrors reality: `/auth/refresh` returns only tokens, while `register` and `login` return tokens plus the user. Rather than duplicate the two token fields, `AuthResponseDto` extends `TokensDto`.

`UserSummaryDto` is the more important idea. It lists exactly what a client may see: id, email, name. Not `password`. Not `hashedRefreshToken`. Returning the raw `User` entity would leak both — an **allow-list** beats remembering to strip fields, because the failure mode of forgetting is silence.

6.6 The strategies — where tokens are verified

`src/modules/auth/strategies/jwt.strategy.ts`

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(
    configService: ConfigService,
    private readonly usersService: UsersService,
  ) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: configService.getOrThrow<string>('JWT_ACCESS_SECRET'),
    });
  }

  // Whatever this returns becomes `request.user`.
  async validate(payload: JwtPayload): Promise<AuthenticatedUser> {
    // The token could still point at a user that was deleted since it was issued.
    const user = await this.usersService.findById(payload.sub);
    if (!user) {
      throw new UnauthorizedException('User no longer exists');
    }

    return { id: user.id, email: user.email, name: user.name };
  }
}
```

PassportStrategy(Strategy, 'jwt')	Registers under the name `'jwt'`. `JwtAuthGuard` asks for that name. **The original code omitted this name on both strategies** — see Part 10.
fromAuthHeaderAsBearerToken()	Read the token from `Authorization: Bearer <token>`.
ignoreExpiration: false	Enforce `exp`. Setting this to `true` would make expiry advisory — and quietly undo the entire short-lifetime design.
secretOrKey	The access secret specifically. Not the refresh secret. This is the line that keeps the two token types apart.

The database lookup in `validate()` is a deliberate cost. A pure JWT approach would trust the payload and skip it — but then a deleted or banned user would keep working until their token expired. One indexed lookup by primary key is cheap; the alternative is a security hole with a fifteen-minute tail.

The refresh strategy — reads the token from the cookie

The refresh token does not travel in a header or a body. It lives in an `httpOnly` cookie (Part 3), so this strategy pulls it out of the cookie instead of the `Authorization` header.

src/modules/auth/strategies/jwt-refresh.strategy.ts

```
/** Pull the refresh token out of the httpOnly cookie (never a header, never the body). */
function refreshTokenFromCookie(request: Request): string | null {
  const cookies = request.cookies as Record<string, string> | undefined;
  return cookies?.[REFRESH_COOKIE] ?? null;
}

@Injectable()
export class JwtRefreshStrategy extends PassportStrategy(Strategy, 'jwt-refresh') {
  constructor(configService: ConfigService) {
    super({
      jwtFromRequest: refreshTokenFromCookie,
      ignoreExpiration: false,
      secretOrKey: configService.getOrThrow<string>('JWT_REFRESH_SECRET'),
      passReqToCallback: true,
    });
  }

  validate(request: Request, payload: JwtPayload): RefreshRequestUser {
    const refreshToken = refreshTokenFromCookie(request);

    if (!refreshToken) {
      throw new UnauthorizedException('Refresh token is missing');
    }

    return { userId: payload.sub, refreshToken };
  }
}
```

Three things make it work:

1. **A different secret.** `JWT_REFRESH_SECRET`. An access token presented here fails signature verification.
2. **A cookie extractor.** `refreshTokenFromCookie` reads `request.cookies[REFRESH_COOKIE]` — which only has a value because `cookieParser()` ran first (see `main.ts`).
3. **`passReqToCallback: true`.** This gives `validate()` the raw request, so we can read the same cookie value back out to hand to the service.

Why do we need the raw string? Because Passport hands us the *decoded payload*, and the payload is not enough. We still have to compare this token against the hash stored on the user's row. **A valid signature only proves the token was issued by us — not that it is still the current one.** That distinction is the whole of rotation and revocation.

Where the cookie itself is defined: `refresh-cookie.ts`

The cookie's name and options live in one small file so "set" and "clear" cannot drift apart.

src/modules/auth/refresh-cookie.ts

```
export const REFRESH_COOKIE = 'refresh_token';

export function refreshCookieOptions(configService: ConfigService): CookieOptions {
  const isProd = configService.get<string>('NODE_ENV') === 'production';
  return {
    httpOnly: true,                       // JavaScript can never read it
    secure: isProd,                       // HTTPS-only in prod; off on http://localhost
    sameSite: isProd ? 'none' : 'lax',    // cross-site in prod needs 'none'
    path: '/auth',                        // only sent to /auth/* requests
    maxAge: 7 * 24 * 60 * 60 * 1000,      // 7 days, matches the refresh lifetime
  };
}
```

httpOnly	The whole point — an XSS bug on the site cannot read the cookie, so it cannot steal the refresh token.
secure	Off in dev because localhost is plain http; on in prod so the cookie only rides over HTTPS.
sameSite	In prod the SPA is a different origin, so the cookie is cross-site and needs `'none'` (which requires `secure`). In dev we assume a same-origin dev proxy, so `'lax'` is enough.
path '/auth'	The browser only attaches the cookie to `/auth/*` calls, so it never rides along on unrelated API requests.

6.7 The guards

`src/modules/auth/guards/`

```
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}

@Injectable()
export class JwtRefreshGuard extends AuthGuard('jwt-refresh') {}

@Injectable()
export class GoogleAuthGuard extends AuthGuard('google') {}
```

Three one-line classes, and that is genuinely all they need to be. Each is a named handle a controller can point at. Their entire behaviour is inherited: run the named strategy, attach the result to `request.user`, or throw 401. `GoogleAuthGuard` is the new one — on the first Google route it redirects to Google, on the callback it fills `request.user`.

The Google strategy

`src/modules/auth/strategies/google.strategy.ts`

```
@Injectable()
export class GoogleStrategy extends PassportStrategy(Strategy, 'google') {
  constructor(configService: ConfigService) {
    super({
      clientID: configService.getOrThrow<string>('GOOGLE_CLIENT_ID'),
      clientSecret: configService.getOrThrow<string>('GOOGLE_CLIENT_SECRET'),
      callbackURL: configService.getOrThrow<string>('GOOGLE_CALLBACK_URL'),
      scope: ['email', 'profile'],
    });
  }

  validate(_accessToken, _refreshToken, profile: Profile, done: VerifyCallback): void {
    const email = profile.emails?.[0]?.value;
    if (!email) {
      done(new Error('Google account has no email'), undefined);
      return;
    }
    done(null, { email, name: profile.displayName });   // becomes request.user
  }
}
```

It reads like the JWT strategies but the credential is different: instead of a token, Passport handles the whole OAuth dance (redirect to Google, user consents, Google returns a code, Passport swaps the code for the profile) and then calls `validate()`. We keep only the email and name. The two token arguments Google gives us are Google's own tokens — we ignore them, because we issue our own (that is what `_accessToken` / `_refreshToken` with the underscore means: deliberately unused).

> **TWO CALLBACK URLS, DO NOT CONFUSE THEM**
>
> `GOOGLE_CALLBACK_URL` (`/auth/google/callback`) is where *Google* sends the user after consent — it must be registered in the Google Cloud Console exactly. `FRONTEND_URL` is where *we* send the user after we finish — the SPA. One is Google → us; the other is us → the frontend.

6.8 `src/modules/auth/auth.service.ts` — the core

Every rule lives here. No HTTP knowledge — it takes plain arguments and returns plain objects, which is exactly why it is easy to test.

register

```
async register(createUserDto: CreateUserDto): Promise<AuthResponseDto> {
  const alreadyRegistered = await this.usersService.findByEmail(createUserDto.email);
  if (alreadyRegistered) {
    throw new ConflictException('Email already registered');
  }

  const user = await this.usersService.create(createUserDto);
  const tokens = await this.issueTokens(user);

  return { ...tokens, user: toUserSummary(user) };
}
```

Registration logs you straight in. That is a UX decision — the alternative, redirecting to a login form immediately after signup, asks the user to type credentials they entered five seconds ago.

`ConflictException` maps to HTTP 409. Nest's built-in exception classes each carry their status code, so you throw a meaning and the framework handles the protocol.

login

```
async login(loginDto: LoginDto): Promise<AuthResponseDto> {
  const user = await this.usersService.validateCredentials(
    loginDto.email,
    loginDto.password,
  );
  // Same message for "no such email" and "wrong password" on purpose: we do
  // not want to tell an attacker which emails exist.
  if (!user) {
    throw new UnauthorizedException('Invalid email or password');
  }

  const tokens = await this.issueTokens(user);
  await this.usersService.updateLastLogin(user.id);

  return { ...tokens, user: toUserSummary(user) };
}
```

> **"INVALID EMAIL OR PASSWORD" IS VAGUE ON PURPOSE**
>
> A friendlier message — "no account with that email" — is a free account enumeration oracle. An attacker feeds in a leaked email list and learns which addresses have accounts here, then targets those with credential stuffing. The deliberately vague message is the counterpart to the timing defence in §6.4: both close the same leak, one via content and one via timing. Closing only one is closing neither.

loginWithGoogle

```
async loginWithGoogle(googleUser: GoogleUser): Promise<AuthResponseDto> {
  const user = await this.usersService.findOrCreateGoogleUser(googleUser);
  const tokens = await this.issueTokens(user);
  await this.usersService.updateLastLogin(user.id);

  return { ...tokens, user: toUserSummary(user) };
}
```

Compare it to `login` above: the only difference is the first line. Password login verifies a password; Google login trusts Google's word and find-or-creates the account. From `issueTokens` onward they are *identical* — same tokens, same refresh hash, same everything. This is the "Google is only the front door" idea made concrete: one shared code path once identity is established.

refreshTokens — the most important method in the codebase

```
async refreshTokens(userId: string, refreshToken: string): Promise<TokensDto> {
  const user = await this.usersService.findById(userId);

  // No stored hash means the user logged out — the signature may still be
  // valid, but we revoked the token.
  if (!user || !user.hashedRefreshToken) {
    throw new UnauthorizedException('Refresh token is no longer valid');
  }

  if (hashToken(refreshToken) !== user.hashedRefreshToken) {
    throw new UnauthorizedException('Refresh token is no longer valid');
  }

  return this.issueTokens(user);
}
```

Three gates, in order:

1. **Does the user still exist?** Deleted accounts cannot refresh.
2. **Is there a stored hash at all?** `NULL` means logged out. This is what makes logout real — the JWT signature is still perfectly valid here, and we reject it anyway.
3. **Does this token match the current one?** If not, it is an old rotated-away token. Dead.

Then `issueTokens` runs again, overwriting the stored hash — which is what makes the token just used single-use.

issueTokens

```
private async issueTokens(user: User): Promise<TokensDto> {
  const payload: JwtPayload = { sub: user.id, email: user.email };

  const [accessToken, refreshToken] = await Promise.all([
    this.jwtService.signAsync(payload, {
      secret: this.configService.getOrThrow<string>('JWT_ACCESS_SECRET'),
      expiresIn: this.expiry('JWT_ACCESS_EXPIRES_IN', '15m'),
    }),
    this.jwtService.signAsync(payload, {
      secret: this.configService.getOrThrow<string>('JWT_REFRESH_SECRET'),
      expiresIn: this.expiry('JWT_REFRESH_EXPIRES_IN', '7d'),
      // "iat" only has second precision, so two refreshes inside the same
      // second would produce byte-identical tokens. A random id keeps every
      // refresh token unique.
      jwtid: randomUUID(),
    }),
  ]);

  await this.usersService.setHashedRefreshToken(user.id, hashToken(refreshToken));

  return { accessToken, refreshToken };
}
```

Same payload, two different secrets, two different lifetimes. `Promise.all` signs both concurrently rather than one after the other — signing is CPU work, and there is no reason to serialise it.

The last line is where statefulness enters: the refresh token's hash is written to the user's row. Every call to this method silently invalidates the previous refresh token. Rotation is not a separate feature; it is a consequence of `issueTokens` always writing.

`jwtid` — a subtle bug, prevented

JWT's `iat` claim has one-second resolution. Two tokens signed for the same user, with the same payload, the same secret and the same expiry, within the same second, are **byte-for-byte identical**.

For a fast client that refreshes twice in a second, the "new" token would equal the old one. Functionally the hash still matches so nothing breaks — but the guarantee "each refresh token is distinct" would be quietly false, and the rotation test would fail intermittently. A random `jti` makes every token unique by construction.

hashToken — and why not bcrypt

```
function hashToken(token: string): string {
  return createHash('sha256').update(token).digest('hex');
}
```

> **THE TRAP THIS AVOIDS**
>
> The instinct is to reuse bcrypt — it is already imported, and it is what we use for passwords. **It would be a real vulnerability here.**
>
> bcrypt silently ignores everything past the first **72 bytes** of its input. A JWT is 200–400 characters. So bcrypt would only ever see the first 72 — and for two refresh tokens belonging to the same user, those first 72 bytes are the header plus the start of the payload: *nearly identical*. Two genuinely different tokens could hash to the same value and compare as equal, defeating rotation entirely.
>
> SHA-256 hashes the whole input. And the slowness of bcrypt buys nothing here anyway: bcrypt is slow to defend low-entropy human passwords against brute force. A signed JWT is long and high-entropy — nobody is brute-forcing it. A fast hash over the complete input is both correct and appropriate.

logout

```
async logout(userId: string): Promise<{ message: string }> {
  await this.usersService.setHashedRefreshToken(userId, null);
  return { message: 'Logged out successfully' };
}
```

One line of real work. Setting the stored hash to `NULL` means the next refresh attempt hits gate 2 in `refreshTokens` and fails.

> **WHAT LOGOUT DOES NOT DO**
>
> The user's current **access token still works** until it expires — at most fifteen minutes. Nothing checks access tokens against the database, by design; that check is what makes them fast.
>
> This is the accepted trade-off of the entire pattern, and it is why the access token lifetime is short. If you need instant global revocation you need a blocklist consulted on every request — which is a session system wearing a JWT costume, and pays the per-request database cost we chose to avoid.

6.9 `src/modules/auth/auth.controller.ts`

The controller translates HTTP into a service call. The one extra job it has now is splitting the tokens across two channels: access token into the JSON body, refresh token into the `httpOnly` cookie.

```
@Controller('auth')
export class AuthController {
  constructor(
    private readonly authService: AuthService,
    private readonly configService: ConfigService,
  ) {}

  @Post('register')
  @HttpCode(HttpStatus.CREATED)
  async register(
    @Body() createUserDto: CreateUserDto,
    @Res({ passthrough: true }) response: Response,
  ): Promise<AuthBodyDto> {
    const { accessToken, refreshToken, user } =
      await this.authService.register(createUserDto);
    this.setRefreshCookie(response, refreshToken);
    return { accessToken, user };   // NOTE: no refreshToken in the body
  }

  @Post('login')  /* ...same shape as register... */

  @Post('refresh')
  @UseGuards(JwtRefreshGuard)
  async refresh(
    @CurrentUser() user: RefreshRequestUser,
    @Res({ passthrough: true }) response: Response,
  ): Promise<AccessTokenBodyDto> {
    const { accessToken, refreshToken } =
      await this.authService.refreshTokens(user.userId, user.refreshToken);
    this.setRefreshCookie(response, refreshToken);   // rotate the cookie too
    return { accessToken };
  }

  @Post('logout')
  @UseGuards(JwtAuthGuard)
  async logout(
    @CurrentUser() user: AuthenticatedUser,
    @Res({ passthrough: true }) response: Response,
  ) {
    const result = await this.authService.logout(user.id);
    response.clearCookie(REFRESH_COOKIE, {
      ...refreshCookieOptions(this.configService),
      maxAge: undefined,
    });
    return result;
  }

  // Google routes: GET /auth/google and GET /auth/google/callback — see below.
  // GET /auth/me — unchanged, returns request.user.

  /** Write the refresh token into the httpOnly cookie. */
  private setRefreshCookie(response: Response, refreshToken: string): void {
    response.cookie(
      REFRESH_COOKIE,
      refreshToken,
      refreshCookieOptions(this.configService),
    );
```

```
  }
}
```

Piece	What it does
`@Res({ passthrough: true })`	Gives the handler the Express response so it can set cookies — but `passthrough` means Nest still sends the returned object as JSON. Without `passthrough`, returning a value would be ignored.
`setRefreshCookie`	One private helper, called by register / login / refresh, so the cookie is written the same way every time.
`clearCookie`	Logout wipes the cookie. It must repeat the same `path` / `sameSite` / `secure`, or the browser treats it as a different cookie and keeps the old one.
return `{ accessToken, user }`	The refresh token is deliberately absent from every body. It lives only in the cookie.

`GET /auth/me` is unchanged: its body is `return user`. All the work happened in the guard — it verified the token, loaded the user, and attached it. The endpoint lets a frontend answer "is my access token still good, and who does it belong to?" on page load.

The Google routes

```
@Get('google')
@UseGuards(GoogleAuthGuard)
googleAuth(): void {
  // empty — GoogleAuthGuard redirects the browser to Google
}

@Get('google/callback')
@UseGuards(GoogleAuthGuard)
async googleCallback(
  @CurrentUser() googleUser: GoogleUser,
  @Res() response: Response,
): Promise<void> {
  const { accessToken, refreshToken } =
    await this.authService.loginWithGoogle(googleUser);
  this.setRefreshCookie(response, refreshToken);

  const frontendUrl = this.configService.getOrThrow<string>('FRONTEND_URL');
  response.redirect(`${frontendUrl}/oauth/callback#accessToken=${accessToken}`);
}
```

The first route is an empty method — the guard does the redirect, so the body never runs. The callback uses plain `@Res()` (no `passthrough`) because it takes full control and issues a `redirect` instead of returning JSON. The refresh token still goes into the cookie; only the access token rides the fragment.

6.10 `src/common/`

`src/common/interfaces/jwt-payload.interface.ts`

```
export interface JwtPayload {
  sub: string;
  email: string;
}

/** What the jwt strategy attaches to `request.user` on protected routes. */
export interface AuthenticatedUser {
  id: string;
  email: string;
  name?: string;
}

/** What the jwt-refresh strategy attaches to `request.user` on POST /auth/refresh. */
export interface RefreshRequestUser {
  userId: string;
  refreshToken: string;
}
```

Three interfaces, three distinct shapes, deliberately not merged. The refresh route's `request.user` genuinely is a different thing from a protected route's, and giving each a name means TypeScript catches you if you confuse them.

src/common/decorators/current-user.decorator.ts

```
export const CurrentUser = createParamDecorator(
  (_data: unknown, context: ExecutionContext) => {
    const request = context
      .switchToHttp()
      .getRequest<Request & { user: unknown }>();
    return request.user;
  },
);
```

A custom parameter decorator. It replaces this:

```
logout(@Request() req: any) {
  return this.authService.logout(req.user.id);   // `any` — no type safety at all
}
```

with this:

```
logout(@CurrentUser() user: AuthenticatedUser) {
  return this.authService.logout(user.id);       // fully typed
}
```

Shorter, and — more importantly — typed. `req.user` is `any`, so a typo like `req.user.userId` compiles fine and fails at runtime. With the decorator the compiler catches it.

6.11 The modules

src/modules/auth/auth.module.ts

```
@Module({
  imports: [
    PassportModule,
    // Empty on purpose: access and refresh tokens use different secrets and
    // lifetimes, so AuthService passes those per sign() call instead.
    JwtModule.register({}),
    UsersModule,
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy, JwtRefreshStrategy],
  exports: [AuthService],
})
export class AuthModule {}
```

imports	Other modules whose exports we need. `UsersModule` is here because `AuthService` uses `UsersService`.
providers	Classes this module creates. The two strategies are listed here purely so Nest instantiates them — that act is what registers them with Passport.
controllers	Route handlers this module owns.
exports	What other modules may use. Anything not exported is private to this module.

`JwtModule.register({})` is intentionally empty. The usual tutorial puts the secret and expiry here — which works only when there is one kind of token. We have two, with different secrets. So the module provides the signing machinery and `AuthService` supplies the parameters per call.

src/modules/users/users.module.ts

```
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService], // AuthModule needs it
})
export class UsersModule {}
```

`forFeature([User])` makes the `User` repository injectable inside this module. Note what is *not* exported: the repository. Only `UsersService` leaves this module, which is what enforces "one module owns its table" from Part 5.

7 The Six Flows, Step by Step

Following a real request through every file it touches.

7.1 Register POST /auth/register

```
POST /auth/register
Content-Type: application/json

{ "email": "jawad@example.com", "password": "password123", "name": "Jawad" }
```

1. **Nest matches the route.** `@Controller('auth')` + `@Post('register')`. No guard, so nothing blocks it.
   `auth.controller.ts`
2. **ValidationPipe checks the body** against `CreateUserDto`. Email well-formed? Password ≥ 8 characters? Any undeclared fields? A failure here returns **400** and the controller never runs.
   `main.ts → create-user.dto.ts`
3. **Controller calls the service.** One line: `return this.authService.register(createUserDto)`.
   `auth.controller.ts`
4. **Duplicate check.** `findByEmail` — lowercased first. If a row comes back, throw `ConflictException` → **409**.
   `auth.service.ts → users.service.ts`
5. **Hash the password.** `bcrypt.hash(password, 10)`, roughly 80 ms of deliberate work. The plain password is never stored.
   `users.service.ts`
6. **Insert the row.** TypeORM generates a UUID and writes to `users`.
   `users.service.ts`
7. **Sign both tokens** in parallel — access with one secret for 15m, refresh with the other for 7d plus a random `jti`.
   `auth.service.ts → issueTokens`
8. **Store the refresh hash.** SHA-256 of the refresh token onto the user's row. The account is now in a refreshable state.
   `users.service.ts → setHashedRefreshToken`
9. **Set the refresh cookie.** The controller writes the refresh token into the `httpOnly` cookie via `Set-Cookie`, then drops it from the body.
   `auth.controller.ts → setRefreshCookie`
10. **Return 201** with the access token and user summary. The refresh token is in the cookie, never the body; the password is never anywhere.
    `auth.controller.ts`

```
201 Created
Set-Cookie: refresh_token=eyJhbGci...; Max-Age=604800; Path=/auth; HttpOnly; SameSite=Lax

{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "id": "b3f1c2d4-...", "email": "jawad@example.com", "name": "Jawad" }
}
```

Notice the two channels: the access token in the JSON body (frontend puts it in memory), the refresh token in the `Set-Cookie` header (the browser stores it, JavaScript cannot touch it).

7.2 Login POST /auth/login

1. **Validate the body** against `LoginDto` — a valid email and a non-empty password.
   `login.dto.ts`
2. **Look up and compare.** `validateCredentials` finds the user and runs `bcrypt.compare(typed, stored)`. If the email is unknown it compares against `DUMMY_HASH` instead, so both paths take the same time.
   `users.service.ts`
3. **Reject uniformly.** Either failure → **401**, message "Invalid email or password". The client cannot tell which failed.
   `auth.service.ts`
4. **Issue a fresh pair** and overwrite the stored refresh hash. Logging in again invalidates the previous session's refresh token — see the note below.
   `auth.service.ts → issueTokens`
5. **Set the refresh cookie**, stamp `lastLoginAt`, and return **200** with the access token and user (refresh token in the cookie, as with register).
   `auth.controller.ts → users.service.ts`

> **ONE REFRESH SLOT PER USER**
>
> There is a single `hashedRefreshToken` column, so a user has one refreshable session. Log in on your phone and your laptop session can no longer refresh — it dies when its access token expires, up to fifteen minutes later.
>
> Fixing this means a separate `refresh_tokens` table with one row per device. It was deliberately left out (Part 13) as unnecessary complexity for the current stage, and it is the first thing to add if multi-device matters.

7.3 A protected request GET /auth/me access token

```
GET /auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...   ← the ACCESS token
```

1. **JwtAuthGuard fires** before anything else and asks Passport for the strategy named `'jwt'`.
   `jwt-auth.guard.ts`

2. **Passport extracts the token** from the `Authorization` header. Missing or malformed → **401**.
   `passport-jwt`
3. **Passport verifies the signature** using `JWT_ACCESS_SECRET`, and checks `exp`. Tampered or expired → **401**. Our code has not run yet.
   `passport-jwt`
4. **`validate(payload)` runs** and looks the user up by `payload.sub`. Deleted since the token was issued → **401**.
   `jwt.strategy.ts`
5. **The return value becomes `request.user`** — id, email, name.
   `jwt.strategy.ts`
6. **`@CurrentUser()` reads it** and passes it in as a typed parameter.
   `current-user.decorator.ts`
7. **The controller returns it. 200**. No database work in the handler itself.
   `auth.controller.ts`

> **THE PATTERN TO COPY FOR EVERY FUTURE ENDPOINT**
>
> Every protected route in FinSight — expenses, budgets, reports — follows exactly this shape. Add `@UseGuards(JwtAuthGuard)` and `@CurrentUser()`, and you have an authenticated user id to scope queries by. Nothing about authentication needs to be rewritten per feature.

7.4 Refresh POST /auth/refresh refresh cookie

The access token expired. Rather than sending the user back to the login form, the frontend quietly calls `/auth/refresh` — and it does not even have to attach the refresh token, because the browser sends the cookie automatically.

```
POST /auth/refresh
Cookie: refresh_token=eyJhbGciOiJIUzI1NiIs...   ← sent automatically by the browser
(no Authorization header, no body)
```

1. **JwtRefreshGuard fires** and runs the strategy named `'jwt-refresh'`.
   `jwt-refresh.guard.ts`
2. **Read the token from the cookie** and verify against `JWT_REFRESH_SECRET`. No cookie → **401**. An access token dropped into the cookie fails the signature check → **401**.
   `jwt-refresh.strategy.ts`
3. **The raw token is recovered** from the cookie — `passReqToCallback` makes the request available. `request.user` becomes `{ userId, refreshToken }`.
   `jwt-refresh.strategy.ts`
4. **Load the user.** Gone → **401**.
   `auth.service.ts → refreshTokens`

5. **Is `hashedRefreshToken` NULL?** Then the user logged out. **401** — even though the signature was perfectly valid. This single check is what makes logout meaningful.
   `auth.service.ts`

6. **Compare hashes.** SHA-256 the presented token; mismatch means it is an old rotated-away token → **401**.
   `auth.service.ts → hashToken`

7. **Issue a new pair, overwrite the stored hash, set the new refresh cookie.** The old cookie value is now dead. **200**, new access token in the body.
   `auth.service.ts → auth.controller.ts`

How the frontend uses this

```
Axios / fetch response interceptor (all calls use credentials: 'include'):

  1. Request fails with 401
  2. POST /auth/refresh          → the browser attaches the refresh cookie for you
  3. Refresh succeeded?          → store the NEW access token in memory, retry
  4. Refresh failed (401)?       → clear the in-memory access token, redirect to /login
```

Done properly this is invisible. The user never sees the 401, never sees the refresh, and stays logged in for a week. The frontend never touches the refresh token — it just has to send `credentials: 'include'` so the cookie rides along.

7.5 Logout `POST /auth/logout` `access token`

1. **JwtAuthGuard fires** — note this route takes the *access* token, because logging out is an ordinary authenticated action.
   `jwt-auth.guard.ts`

2. **Set the stored hash to NULL.** One UPDATE.
   `auth.service.ts → setHashedRefreshToken(id, null)`

3. **Clear the cookie.** The response sends `Set-Cookie: refresh_token=; Path=/auth`, so the browser deletes it.
   `auth.controller.ts → response.clearCookie`

4. **Return 200.** The refresh token can no longer be redeemed; the access token remains valid for its remaining minutes.
   `auth.controller.ts`

```
before logout        hashedRefreshToken = "7f83b1657ff1fc53b92dc1815…"
                     refresh works ✓        access works ✓

after logout         hashedRefreshToken = NULL   +   cookie cleared
                     refresh → 401 ✗        access still works until exp (≤15 min)

after 15 minutes     fully logged out
```

Logout closes the long-lived door immediately; the short-lived one closes on its own.

Two defences run together: the server sets the stored hash to NULL (so even a copied cookie value is dead), and it clears the browser's cookie. Server-side revocation and cookie deletion are complementary — neither alone is sufficient.

7.6 Sign in with Google `GET /auth/google`

Google verifies who the user is; then we hand out *our own* tokens, so from that point on a Google user is identical to a password user.

```
Browser ⟶ GET /auth/google                (a normal link, not fetch)
Google  ⟶ consent screen ⟶ GET /auth/google/callback?code=...
Server  ⟶ redirect to  FRONTEND_URL/oauth/callback#accessToken=...
```

1. **GoogleAuthGuard fires on** `/auth/google` and redirects the browser to Google's consent screen. The controller method body never runs.
   `google-auth.guard.ts → google.strategy.ts`

2. **Google sends the user back** to `/auth/google/callback` with a one-time `code`. The guard runs again.
   `google.strategy.ts`

3. **Passport swaps the code for the profile** and calls `validate()`, which extracts the email and name into `request.user`.
   `google.strategy.ts`

4. **Find or create the account.** First Google sign-in creates a user with `provider = 'google'` and no password; later ones reuse it. An existing email (even a password account) is simply logged in.
   `auth.service.ts → users.service.ts`

5. **Issue our tokens** — the very same `issueTokens` as everything else — and set the refresh cookie.
   `auth.service.ts → auth.controller.ts`

6. **Redirect to the frontend** with the access token in the URL *fragment* (`#accessToken=…`). The SPA reads it into memory; the refresh token is already in the cookie.
   `auth.controller.ts`

THE ONE IDEA THAT MAKES GOOGLE SIMPLE

Google is only the front door. Once the callback finishes, nothing downstream — refresh, rotation, logout, `/auth/me` — knows or cares that the user came via Google. That is why `loginWithGoogle` reuses the exact `issueTokens` and `setRefreshCookie` that password login uses. Fewer moving parts, one thing to explain.

WHY THE ACCESS TOKEN RIDES ON THE FRAGMENT (#)

A redirect cannot carry a JSON body, so the access token has to travel in the URL. The part after `#` — the fragment — is never sent to any server and never written to server logs or the `Referer` header, so it is the safest place to put a short-lived token. The SPA reads it from `window.location.hash` into memory and clears it from the URL. The refresh token never appears here — it is set as the cookie during the callback.

8 Architecture Decisions & Rejected Alternatives

Fourteen decisions, what else was considered, and why it lost.

8.1 JWT rather than server-side sessions

Decision	Stateless JWT bearer tokens.
Alternative	Session ID in a cookie, session data in Postgres or Redis.
Why	The frontend is a separate SPA on another origin. Cookies across origins bring CORS credentials, `SameSite` configuration, and CSRF defences. Bearer tokens need none of that and work identically for a future mobile client.
Cost accepted	Access tokens cannot be revoked. Mitigated by the 15-minute lifetime and by making the refresh token stateful.

8.2 Two tokens rather than one

Decision	15-minute access token + 7-day refresh token.
Alternative	One token with a middling lifetime, say 24 hours.
Why	A middling lifetime is the worst of both: still long enough that a theft hurts, still short enough to annoy. Splitting the roles lets each be optimised for its own exposure profile.

8.3 Two different secrets

Decision	`JWT_ACCESS_SECRET` ≠ `JWT_REFRESH_SECRET`.
Alternative	One secret, distinguishing the tokens by a `type` claim in the payload.
Why	A claim-based check is a check you can forget to write. Separate secrets make misuse impossible at the cryptographic layer rather than the application layer — the wrong token simply fails signature verification, with no code of ours involved.

8.4 Storing a hash of the refresh token

Decision	SHA-256 of the refresh token on the user row.
Alternative A	Store nothing — trust the signature. Then logout is fiction and rotation is impossible.
Alternative B	Store the token in plain text. A database leak then hands the attacker working credentials, not just hashes.
Why	Hashing gives revocation and rotation while keeping the stored value useless to anyone who steals the database.

8.5 SHA-256 for tokens, bcrypt for passwords

Decision	Different hash functions for the two jobs.
Alternative	bcrypt for both — it is already imported.
Why	**bcrypt truncates at 72 bytes.** JWTs are far longer, and two tokens for the same user share a near-identical first 72 bytes — so different tokens could compare equal, silently breaking rotation. Separately, bcrypt's slowness exists to protect low-entropy human passwords; a signed JWT is high-entropy and needs no such protection.

8.6 A random `jti` on refresh tokens

Decision	`jwtid: randomUUID()` on the refresh token only.
Problem solved	`iat` has one-second resolution, so two refreshes inside the same second produce byte-identical tokens.
Why refresh only	Access tokens are never compared for equality, so uniqueness buys nothing there — and a shorter token is marginally cheaper on every single request.

8.7 Looking the user up on every protected request

Decision	`JwtStrategy.validate()` queries the database.
Alternative	Trust the payload; skip the query entirely. Faster, and fully stateless.
Why	A deleted or banned user would otherwise keep full access until their token expired. One lookup on a primary key is a few hundred microseconds. Correctness wins at this scale; revisit with a short-TTL cache if it ever shows up in a profile.

8.8 Registration returns tokens immediately

Decision	Sign up and you are logged in.
Alternative	Create the account, return 201, force a login.
Why	The user typed those exact credentials moments ago. Asking again is friction with no security benefit — we just verified them by construction.

8.9 Identical failure message for both login failures

Decision	"Invalid email or password" for both cases, plus equalised timing.
Alternative	Specific messages — friendlier UX.
Why	Specific messages are an account-enumeration oracle. And a vague message with unequal timing leaks the same information through a different channel, so both halves are required or neither is worth doing.

8.10 Relative imports, no path aliases

Decision	`../users/users.service`.
Alternative	`@users/users.service` — what the original code attempted.
Why	Aliases are a compile-time feature. The emitted JavaScript keeps the alias string, which Node cannot resolve, so the app needs an extra runtime resolver. The original had neither the resolver nor the `paths` entry — it failed at both compile and run time.

8.11 The service knows nothing about HTTP

Decision	`AuthService` takes plain arguments, returns plain objects, and never touches `Request` or `Response`.
Alternative	Pass the request object down into the service.
Why	Testability, mostly. Unit-testing the service needs no HTTP server and no mock request — you call a method with two strings. It also means the logic would work unchanged behind a GraphQL resolver or a CLI command.

8.12 Refresh token in an httpOnly cookie, not the JSON body

Decision	The refresh token is set as an `httpOnly` cookie and never returned in any response body. The access token stays in the body.
Alternative	Return both tokens in the body and let the frontend store them (usually in `localStorage`).
Why	A refresh token in `localStorage` is readable by any XSS on the site, and it is the long-lived, valuable token. `httpOnly` puts it out of JavaScript's reach entirely. The access token can stay in the body because it is short-lived and the frontend needs to attach it as a header anyway.
Cost accepted	Cross-origin cookies need CORS `credentials` and `SameSite` care — handled by a dev proxy in development and same-domain in production.

8.13 Google issues its own tokens, not ours reused

Decision	After Google verifies the user, we mint our own access + refresh tokens and ignore Google's.
Alternative	Keep using Google's tokens to authenticate every request.
Why	Reusing Google's tokens would tie every protected route to a Google API call and Google's token format. Minting our own means Google users and password users are identical everywhere downstream — one refresh flow, one logout, one guard. Google is only the front door.

8.14 Access token on the URL fragment after Google login

Decision	The Google callback redirects to `FRONTEND_URL/oauth/callback#accessToken=…` — the token is in the fragment (after `#`).
Alternative	Put it in the query string (`?accessToken=`), or set it as a second cookie.
Why	A redirect cannot carry a JSON body. The query string is sent to the server and written to logs and `Referer`; the fragment is not, so it is the safe half of the URL for a short-lived token. A second cookie would defeat the "access token in memory" model. The refresh token still travels only by httpOnly cookie.

9 Security Decisions

Which attack, and exactly which line stops it.

Attack	How it works	Defence in this code
Database leak	Someone dumps the `users` table and reads credentials.	Passwords are bcrypt hashes with per-user salts; refresh tokens are SHA-256 hashes. Neither can be used to log in.
Rainbow tables	Pre-computed hashes of common passwords, matched in bulk against the dump.	bcrypt's automatic per-user salt makes every hash unique, so bulk matching fails and each user must be attacked separately.
Brute force on a dump	GPU-guessing billions of passwords per second against stolen hashes.	bcrypt cost factor 10 — ~80 ms per guess by design, which turns billions per second into a handful.
Token tampering	Decode the JWT, change `sub` to another user's id, re-encode.	The signature no longer matches and the secret is needed to fix it. Rejected by Passport before our code runs.
Stolen access token	Captured from a log, a proxy, or XSS, then replayed.	Expires in 15 minutes. Damage is bounded by time — this is the reason the lifetime is short.
Stolen refresh token	Replayed to mint fresh access tokens indefinitely.	Rotation. The moment the real user refreshes, the thief's copy no longer matches the stored hash and is dead.
Refresh token stolen by XSS	A script injected into the site reads stored tokens and exfiltrates them.	The refresh token lives in an `httpOnly` cookie — JavaScript cannot read it at all, so XSS cannot reach it. It is never in `localStorage` or any response body.
Refresh token used as an access token	Present the 7-day token to protected routes and enjoy a week of access.	Separate secrets. It fails signature verification at `JwtStrategy`. Asserted by a test.
Password login on a Google account	Try to brute-force a Google user through the password route.	Google accounts have a null password; `validateCredentials` rejects them (and still runs the dummy compare, so timing does not leak).
Logout that does not log out	The user clicks logout; the captured token keeps working.	`hashedRefreshToken = NULL` makes refresh fail immediately. The access token still has ≤15 minutes — a documented, bounded gap.
Account enumeration (content)	Different error messages reveal which emails are registered.	One message for both failures: "Invalid email or password".
Account enumeration (timing)	Same message, but unknown emails answer far faster.	`bcrypt.compare` against `DUMMY_HASH` on the unknown-email path, equalising response time.
Mass assignment	POST extra fields — `{"isAdmin": true}` — hoping they are persisted.	`whitelist` strips undeclared properties; `forbidNonWhitelisted` rejects the request outright with 400.

Password in a response	Returning the raw entity leaks the hash and the stored refresh hash.	Responses are built from `toUserSummary()` — an allow-list of id, email, name. Asserted by an e2e test.
Deleted user keeps access	Account removed, but the issued token stays cryptographically valid.	`JwtStrategy.validate()` loads the user on every request and throws if the row is gone.
Secrets in version control	`.env` is committed; secrets live in git history forever.	`.env` is gitignored. `.env.example` documents the shape with placeholder values.
SQL injection	Crafted input breaks out of the query.	TypeORM parameterises every query. No string concatenation anywhere in this code.

Gaps you should know about

Not defended	What it means, and what to do
Rate limiting	Nothing stops thousands of login attempts per second against one account. This is the most important missing defence. Fix with `@nestjs/throttler` — roughly ten lines.
Password strength	The only rule is eight characters, so `password` passes. Consider a blocklist of common passwords or a strength estimator.
Refresh reuse response	We detect a rotated-away token and return 401, but do not treat it as the theft signal it probably is. A stronger response revokes all of that user's sessions.
HTTPS enforcement	Bearer tokens over plain HTTP are readable by anyone on the network. This is a deployment concern, not a code one — terminate TLS at the proxy and never serve the API over HTTP.

10 Bugs Found in the Original Code

Seven real defects in the previous version, and what each one broke.

The `auth/` folder already contained a partial implementation. It was not merely incomplete — several parts were actively broken in ways that are worth studying, because each is a mistake that is easy to make and hard to spot.

1. The refresh strategy was a copy-paste of the access strategy

the original jwt-refresh.strategy.ts

```
// file: jwt-refresh.strategy.ts
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {   // ← same class name!
  constructor(configService: ConfigService, ...) {
    super({
      secretOrKey: configService.get('JWT_SECRET'),             // ← the ACCESS secret
    });
  }
}
```

Three defects in one file. The class was still named `JwtStrategy`. It registered under Passport's default name `'jwt'` rather than `'jwt-refresh'`. And it verified against the access secret.

Consequences: `JwtRefreshGuard extends AuthGuard('jwt-refresh')` referred to a strategy that did not exist, so `POST /auth/refresh` could never work. Two classes both registering as `'jwt'` meant one silently overwrote the other. And with one shared secret, the entire separate-secrets protection was absent — a refresh token would have functioned as a week-long access token.

> **LESSON**
>
> Copy-paste between two files that must differ is exactly where this class of bug is born. The two strategies look 80% alike, and the 20% that differs is the entire security property.

2. The password field was validated as an email

```
export class LoginDto {
  @IsEmail()
  email !: string;

  @ApiProperty({ description: 'User Password' })
  @IsEmail()          // ← copy-paste; should be @IsString()
  password !: string;
}
```

No password that is not also a valid email address could pass validation. Login was unusable. Interestingly this bug was *invisible in practice* because of bug 3 — with no `ValidationPipe` registered, no validator ran at all, so the wrong decorator never fired. Two bugs cancelling out into an appearance of working.

3. No global ValidationPipe

Every DTO carried `@IsEmail()`, `@MinLength(8)`, `@IsString()` — and none of them ran. class-validator decorators are inert metadata; something has to read them. Without `app.useGlobalPipes(new ValidationPipe())` in `main.ts` the API accepted a two-character password, a malformed email, or arbitrary extra fields.

> **THE DANGEROUS SHAPE OF THIS BUG**
>
> The code *looks* validated. A reviewer scanning the DTOs sees rules everywhere and moves on. There is no error, no warning, no failing test — just an API with no input validation and every appearance of having some.

4. Path aliases that could not resolve

Imports used `@users/users.service`, but `tsconfig.json` had no `paths` entry — so TypeScript could not compile it. Even with the entry added it would still have crashed at runtime, because Nest's default `tsc` build emits the alias string unchanged and Node cannot resolve `@users`. Broken at both stages, for two independent reasons.

5. Filename did not match its imports

The file was `src/users/user.service.ts` (singular). Every import referenced `users.service` (plural). Module not found.

6. `tsconfig.json` had an invalid compiler option

```
tsconfig.json(16,27): error TS5103: Invalid value for '--ignoreDeprecations'.
```

`"ignoreDeprecations": "6.0"` is not accepted by TypeScript 5.9. This one is worth noting because of its scope: `tsconfig.build.json` extends `tsconfig.json`, so `nest build` failed before compiling a single line. Nothing in the project could be built until it was removed.

7. Return types that lied, and unregistered providers

`findByEmail` and `findOne` were typed `Promise<User>` while returning TypeORM's `User | null`. With `strictNullChecks` enabled this fails to compile; without it, every caller believes a user is always present and the app crashes on the first unknown email — precisely the path an attacker probes first.

Separately, `AppController` and `AppService` existed but were never listed in `AppModule`, so `GET /` returned 404 despite the handler being right there in the source.

Summary

Defect	Effect	Now
Refresh strategy duplicated the access strategy	`/auth/refresh` could never work; secrets not separated	Fixed
`@IsEmail()` on the password field	Login impossible once validation ran	Fixed
No global ValidationPipe	Zero input validation, silently	Fixed
Unresolvable `@users/*` aliases	Compile and runtime failure	Relative imports
`user.service.ts` vs `users.service`	Module not found	Fixed
Invalid `ignoreDeprecations`	`nest build` failed entirely	Removed
Return types ignoring `null`	Crash on any unknown email	`User | null`
`AppController` unregistered	`GET /` returned 404	Registered

11 The Tests, and What They Prove

Sixteen tests. No database required.

Two layers, two questions

Layer	File	Question it answers
Unit (7)	`auth.service.spec.ts`	Is the logic correct? Rotation, revocation, duplicate detection, wrong passwords, Google find-or-create.
E2E (9)	`test/auth.e2e-spec.ts`	Is the wiring correct? Do guards actually guard, do pipes actually validate, does the cookie round-trip, are the status codes right.

The split matters because the two layers fail in different ways. Logic can be perfect while the guard is attached to the wrong route — that was literally bug 1 in Part 10, and only an e2e test catches it.

Unit tests: a fake users service

```
class FakeUsersService {
  private users: User[] = [];

  create(dto) { /* push to the array, return it */ }
  findByEmail(email) { /* search the array */ }
  async validateCredentials(email, password) {
    const user = await this.findByEmail(email);
    return user && user.password === password ? user : null;
  }
  async setHashedRefreshToken(id, hash) { /* mutate the array */ }
}
```

An array standing in for Postgres. This is possible only because of dependency injection: `AuthService` declared that it needs *something shaped like* `UsersService`, never that it needs the real one.

Note that passwords are compared as plain strings here. That is deliberate — bcrypt is `UsersService`'s responsibility, and these tests are about `AuthService`. Mixing in real hashing would make them slower and would test the wrong class.

Test	What breaks if it fails
registers a user and returns both tokens	Signup is broken outright
rejects a second registration with the same email	Duplicate accounts
rejects a login with the wrong password	Anyone can log in as anyone
the access token cannot be used as a refresh token	The separate-secrets design has silently collapsed
rotates the refresh token, so the old one stops working	Rotation is not happening; stolen tokens live for 7 days
logout revokes the refresh token	Logout is fiction
Google login creates on first sign-in, reuses on the second	Google would make duplicate accounts, or fail to log returning users in

E2E tests: real HTTP, fake database

```
const moduleFixture = await Test.createTestingModule({
  imports: [ConfigModule.forRoot({ isGlobal: true }), AuthModule],
})
  .overrideProvider(getRepositoryToken(User))
  .useClass(InMemoryUserRepository)     // ← Postgres swapped for an array
  .compile();

app = moduleFixture.createNestApplication();
app.useGlobalPipes(new ValidationPipe({ whitelist: true, ... }));  // same as main.ts
await app.init();
```

This boots the genuine Nest application — real routing, real guards, real strategies, real pipes — with only the repository replaced. Requests go through supertest as actual HTTP.

> **WHY SWAPPING ONLY THE REPOSITORY IS THE RIGHT SEAM**
>
> Everything that could plausibly be miswired is exercised for real. Only the one component that needs external infrastructure is faked. The tests run in about two seconds, need no Postgres, and would still catch a guard pointed at the wrong strategy — which is exactly the bug that was there before.

Test	What it proves
rejects a registration with a short password	The ValidationPipe is actually installed and running — the exact bug from Part 10
registers: access token in body, refresh token only in the cookie	The refresh token never leaks into the JSON body; the password never appears anywhere
sets the refresh cookie as httpOnly	The cookie really carries the `HttpOnly` flag, so JavaScript cannot read it
refuses the profile without a token	`JwtAuthGuard` is really attached, not just imported
refreshes using the cookie, rejects the used cookie afterwards	Cookie-based refresh and rotation work over real HTTP
refuses the refresh route with no cookie	A missing cookie is a clean 401, not a crash
refuses an access token used as the refresh cookie	Separate secrets enforced end to end
logout revokes the refresh token and clears the cookie	Revocation and cookie-clearing survive the full stack
rejects a login with a wrong password	401, not 500, and no information leaked

Running them

```
npm test                              # 7 unit tests + 1 app test
npx jest --config ./test/jest-e2e.json test/auth.e2e-spec.ts   # 9 e2e tests
npx tsc --noEmit                      # type check
npm run build                         # compile
npx eslint "src/**/*.ts"              # lint
```

All of the above currently pass. Note that `test/app.e2e-spec.ts` imports the whole `AppModule` and therefore still needs a live database — it is pre-existing and was not part of this work.

12 Running It

Setup, environment, and a complete walkthrough.

12.1 Environment variables

`.env.example`

```
NODE_ENV=development
PORT=3000

# Postgres
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_USERNAME=postgres
DATABASE_PASSWORD=postgres
DATABASE_NAME=finsight

# JWT — the two secrets MUST be different, that is what stops a refresh token
# from being accepted as an access token.
JWT_ACCESS_SECRET=replace-me-with-a-long-random-string
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_SECRET=replace-me-with-a-different-long-random-string
JWT_REFRESH_EXPIRES_IN=7d

# Google OAuth — from Google Cloud Console → Credentials → OAuth client ID (Web).
# Register this exact callback there under "Authorised redirect URIs".
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

# Where the Google callback sends the user after login (your frontend).
FRONTEND_URL=http://localhost:5173
```

Generate real secrets — never invent them by typing:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

TWO RULES ABOUT THESE SECRETS

1. The two JWT secrets must differ. Identical values silently destroy the protection described in §3 and §8.3, with no error to warn you.

2. `.env` is gitignored and must stay that way. A secret committed once lives in git history forever — rewriting history is painful, and the only real remedy is rotating the secret.

12.2 First run

```
cd fin-sight-backend
npm install

cp .env.example .env          # then edit it: real DB password, real secrets
createdb finsight             # or: CREATE DATABASE finsight;

npm run start:dev
```

On first boot, `synchronize: true` creates the `users` table from the entity. Swagger is then live at `http://localhost:3000/api/docs` — the easiest way to explore the API, since the padlock button lets you paste a token and call protected routes from the browser.

12.3 A full walkthrough with curl

The `-c` and `-b` flags give curl a cookie jar (a file), so it stores and re-sends the refresh cookie exactly like a browser.

Register — save the cookie to a jar

```
curl -X POST http://localhost:3000/auth/register \
  -c jar.txt \
  -H "Content-Type: application/json" \
  -d '{"email":"jawad@example.com","password":"password123","name":"Jawad"}'
```

```
201 Created
Set-Cookie: refresh_token=eyJ...; Path=/auth; HttpOnly; SameSite=Lax   ← saved into jar.txt

{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "user": { "id": "b3f1c2d4-...", "email": "jawad@example.com", "name": "Jawad" }
}
```

Notice: no `refreshToken` in the body. It is in the cookie jar.

Call a protected route

```
curl http://localhost:3000/auth/me \
  -H "Authorization: Bearer <ACCESS_TOKEN>"
```

```
200 OK
{ "id": "b3f1c2d4-...", "email": "jawad@example.com", "name": "Jawad" }
```

Refresh — the cookie is sent from the jar, no token needed

```
curl -X POST http://localhost:3000/auth/refresh \
  -b jar.txt -c jar.txt          # -b sends the cookie, -c saves the rotated one
```

```
200 OK
{ "accessToken": "eyJ...NEW..." }
```

Run it a second time *without* updating the jar (drop `-c`) and you get `401 Unauthorized` — the cookie you replayed was rotated away. That is rotation, observable from the command line.

Google sign-in

```
# Open this in a BROWSER, not curl — it redirects to Google's consent screen:
http://localhost:3000/auth/google
```

After you approve, Google returns you to the callback, and the backend redirects to `FRONTEND_URL/oauth/callback#accessToken=…` with the refresh cookie already set. This step needs real Google credentials in `.env`.

Logout

```
curl -X POST http://localhost:3000/auth/logout \
  -b jar.txt \
  -H "Authorization: Bearer <ACCESS_TOKEN>"
```

The response clears the cookie, and the stored hash is set to NULL. Refreshing again returns `401`. Revocation, observable.

12.4 Every response code, and what it means

Code	Meaning	When you will see it
200	OK	login, refresh, logout, /auth/me
201	Created	register
302	Found (redirect)	the two Google routes — to Google's consent screen, then to the frontend
400	Bad Request	ValidationPipe rejected the body — bad email, short password, or an undeclared field
401	Unauthorized	Wrong credentials; missing, expired, tampered, rotated-away, or revoked token/cookie; wrong token type for the route
409	Conflict	Registering an email that already exists
500	Server Error	Should not happen. If it does, check the database connection and the env vars first.

12.5 Adding your own protected endpoint

Everything above exists so that this is three lines:

```
@Controller('expenses')
export class ExpensesController {
  @Get()
  @UseGuards(JwtAuthGuard)                      // ← 1. require a valid access token
  findMine(@CurrentUser() user: AuthenticatedUser) {   // ← 2. get the typed user
    return this.expensesService.findByUser(user.id);   // ← 3. scope the query
  }
}
```

Import `JwtAuthGuard` and `CurrentUser`, and every future feature is authenticated. Nothing about auth needs to be touched again.

13 What Is Deliberately Not Built

Known gaps — each a decision, not an oversight.

Being explicit about what is missing is more useful than pretending completeness. Everything below was consciously left out, and each entry says when it becomes worth adding.

Missing	What it would take	Add it when
Rate limiting	`@nestjs/throttler`, roughly ten lines of module config.	**Before any public deployment.** This is the most important item on the list — without it, login is brute-forceable.
Multi-device sessions	A `refresh_tokens` table with one row per device, replacing the single column.	As soon as users have both a phone and a laptop. Today, logging in on one kills the other's refresh.
Refresh reuse detection	On a hash mismatch, revoke every session for that user instead of returning a plain 401.	Together with multi-device — the two changes touch the same code.
Email verification	An `emailVerified` flag, a signed one-time link, and an email sender.	When you need working email addresses — for password reset, or to deter throwaway signups.
Password reset	A short-lived single-use token by email, plus a reset endpoint.	Before real users. People forget passwords immediately and permanently.
Roles / permissions	A `role` column, a `RolesGuard`, and a `@Roles()` decorator.	The first time two users need different capabilities. Not before — speculative permission systems age badly.
Password strength rules	A common-password blocklist or a strength estimator in the DTO.	Alongside rate limiting. Eight characters is a weak floor.
Database migrations	TypeORM migrations, and `synchronize: false`.	**Before production.** `synchronize` can drop columns and the data in them.

> **NOW BUILT: COOKIE STORAGE AND GOOGLE SIGN-IN**
>
> Two things this section used to list as "not built" now exist. The refresh token moved into an `httpOnly` cookie (Part 3, §6.6, §8.12), and "Sign in with Google" is fully wired (§6.7, §7.6, §8.13). Both are covered in full above rather than deferred.

> **ON NOT BUILDING THESE YET**
>
> Every item above is genuinely useful and every one adds code that must be maintained, tested, and understood. Building a multi-device session table before anyone uses a second device means carrying that complexity for months to solve a problem nobody has.
>
> The rule this codebase follows: build it when a real requirement asks for it, and until then write down that you chose not to. That written-down decision is what separates a deliberate gap from an oversight — and it is the entire purpose of this section.

Where to go next

1. **Add rate limiting.** Ten lines, and it closes the largest hole.
2. **Wire the frontend interceptor** (§7.4) so refresh is automatic and invisible.
3. **Build the first real feature** — expenses — using the three-line pattern in §12.5. That is the proof the foundation works.
4. **Switch to migrations** before anything goes to production.

FinSight Backend — Authentication Guide

Covers `fin-sight-backend/src/modules/auth`, `src/modules/users`, and `src/common`

All code shown is the code in the repository. All 16 tests, the type check, the build, and lint pass.